

担当箇所の基本設計・詳細設計書_ver 光同期廃止後

25G1113 廣岡花 グループ7

2026年5月13日

1 担当箇所の基本設計

1.1 機能一覧

図1に担当箇所である同期制御機能を中心に詳細化したFBS図を示す。また表1に、各機能の概要と実装上の要点をまとめる。本システムは、1台のPC（Processing）を指揮者UI、1台のMaster Arduinoを制御ハブ、4台のSlave Arduinoを各楽器ノードとして構成し、Processingから受け取った演奏設定をMaster Arduinoが集約して各Slaveへ配信することで、複数ノードの演奏を協調制御する。Processing側では、ユーザーからBPM、オクターブ、演奏順を受け付け、演奏開始後はBPM以外の操作を自動でロックする。また、受信した再生状態に応じてUIロックを解除する。本システムの通信は、演奏開始時に送信される初回一括設定パケットと、演奏中に送信されるテンポパケットの2段階で構成される。初回一括設定パケットでは、オクターブ、演奏順、BPMを8バイトの情報としてProcessingからMasterへ送信し、演奏中はBPMのみを送る。Master Arduinoは受信したBPMから8分音符1つあたりの実時間間隔を算出し、その周期でI2Cバスへテンポパルスを送信する。Slave Arduinoはこのテンポパルスを受信するたびに内部のパルスカウンタを進め、担当順番に応じた待機パルス数に達した時点で演奏状態へ移行し、MIDI音高データをProcessingへ送信する。Processingは受信したデータをもとに発音処理を行う。演奏中の状態遷移はProcessing側のserialEvent()でSTART、FINISHを受信することで管理され、UI操作の可否もこれに連動して制御される。

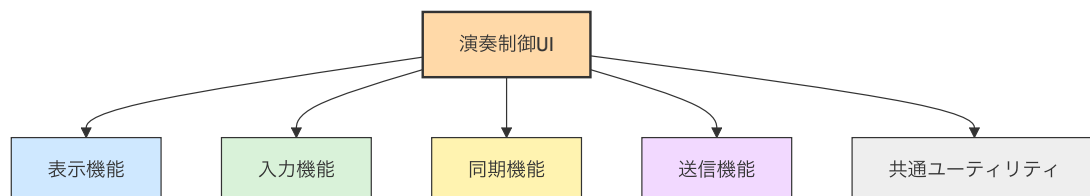


図1 UI面の同期・表示に着目したFBS

表 1 演奏制御 UI の機能一覧

区分	内容
表示機能	タイトルおよび送信ボタンの描画
	BPM 設定エリアの描画
	演奏順番設定エリアの描画
	共通オクターブ設定エリアの描画
	Slave 状態ボックスの描画
	送信パケットおよび状態情報の表示
入力機能	↑/↓キーによる BPM 変更, 1～3 キーによる演奏順登録
	設定送信, 順番リセット, オクターブ変更ボタンの操作
同期機能	START / PLAYING 受信時の UI ロック, FINISH / STOP 受信時の解除
	isPlaying フラグによる操作制御 (演奏順・オクターブ変更禁止, BPM 変更のみ許可)
	シミュレーションモード時の擬似ロックおよび手動解除
送信機能	オクターブ・演奏順・BPM を含む初回設定パケットの生成
	初回設定送信および演奏中の BPM 更新送信
共通ユーティリティ	ボタン領域のホバー判定
	演奏順および選択状態の初期化

1.2 具体的な実装内容

1.2.1 UI 設定入力



図2 実際の Processing の UI 画面

図2に本システムの実際の UI 画面、表2に UI 画面上の主要な操作要素と、それぞれの操作方法および演奏中の挙動を示す。本システムの UI 画面は、BPM 設定エリア、演奏順番設定エリア、共通オクターブ設定エリアの3つの操作領域と、画面右上に配置した設定確定・送信ボタンから構成される。各操作領域は、演奏中であるかどうかを示すフラグ `isPlaying` の状態に応じて編集可否が切り替わり、ロック中は背景色を淡いグレーに変化させるとともに、該当ボタンを非活性表示にすることで、操作可能な項目を視覚的に明示している。BPM 設定エリアでは、キーボードの `[↑]` キーを押すたびに BPM を 10 加算し、`[↓]` キーを押すたびに 10 減算する。BPM は 30 から 180 の範囲に制限されており、この範囲を超える入力は無視される。BPM のみは演奏中であっても変更を許可しており、テンポを動的に調整できる設計としている。演奏順番設定エリアでは、楽器番号に対応するキー (`[1]`: ピアノ, `[2]`: フルート, `[3]`: 木琴) を押した順に演奏順を確定する。ドラムは演奏開始から常時演奏されるため、演奏順設定の対象から除外している。設定した順番は画面中央に「1 番手」「2 番手」「3 番手」の形式で表示され、未選択の楽器は「未選択」と表示される。また、画面右側に配置した順番リセットボタン (クリック操作) を押すことで、演奏順番の選択状態

を初期化できる。演奏中はこの操作領域全体がロックされ、キー入力を受け付けないとともに、リセットボタンの表示も「ロック中」に変化する。共通オクターブ設定エリアでは、画面下部に配置した [▲] ボタンおよび [▼] ボタンのクリック操作によってオクターブを 1 ずつ変更する。オクターブは国際式表記で -1 から 9 の範囲とし、上限・下限を超える操作は無視される。演奏中はオクターブの加減ボタンが非活性表示となり、変更できない状態になる。

表 2 UI 画面における主要操作要素と挙動

UI 要素	操作方法	演奏中の挙動
設定確定・送信ボタン	クリックにより設定をパケット化して Master Arduino へ送信する	常に有効。初回は 8 バイトの初期パケット、2 回目以降は BPM3 バイトの差分パケットを送信する
BPM 設定	[↑] キーで +10, [↓] キーで -10 (範囲: 30~180)	演奏中も変更可能
演奏順番設定	[1]~[3] キーを演奏したい順に押下し、ピアノ・フルート・木琴の演奏順を確定する	ロックされ入力を受け付けない
順番リセットボタン	クリックにより演奏順番の選択状態を初期化する	ボタン表示が「ロック中」に変化し操作不可となる
オクターブ [▲] ボタン	クリックでオクターブを +1 する (上限 9)	非活性表示となり操作不可となる
オクターブ [▼] ボタン	クリックでオクターブを -1 する (下限 -1)	非活性表示となり操作不可となる

1.2.2 Master Arduino へのシリアル送信処理

図 3 に、Processing から Master Arduino への送信処理における状態遷移を示す。Processing 側では、待機中の状態において BPM、演奏順、オクターブの各設定を受け付け、設定確定・送信操作が行われた時点でパケット生成処理へ遷移する。オクターブは -1~9 の範囲で入力し、送信時には 00~10 の 2 桁固定長文字列に変換する。演奏順は楽器 1~3 に対応するキー入力をもとに 3 桁の文字列として保持し、BPM は 30~180 の範囲で 3 桁固定長の値として扱う。これらの値は buildPacket() によりエンコードされ、Master Arduino へ送信可能な形式に整形される。送信処理では、初回送信時と 2 回目以降で通信内容を切り替える。初回送信では、オクターブ、演奏順、BPM を含む 8 バイトの初回設定パケットに改行を付加して送信する。一方、2 回目以降は BPM のみを 3 バイトの差分パケットとして送信し、演奏中の通信負荷を抑制する。この分岐は、初回設定済みかどうかを示すフラグによって管理する。また、Processing 側では Master Arduino からの応答を serialEvent() で受信し、状態に応じて UI ロック制御を行う。Master Arduino から START を受信した場合は演奏中のロック状態へ移行し、演奏順およびオクターブの変更を禁止する。FINISH を受信した場合はロックを解除し、再び待機中の編集可能状態へ戻る。この制御により、演奏中の誤操作を防ぎつつ、BPM のみを動的に変更できる入力インタフェースを実現している。

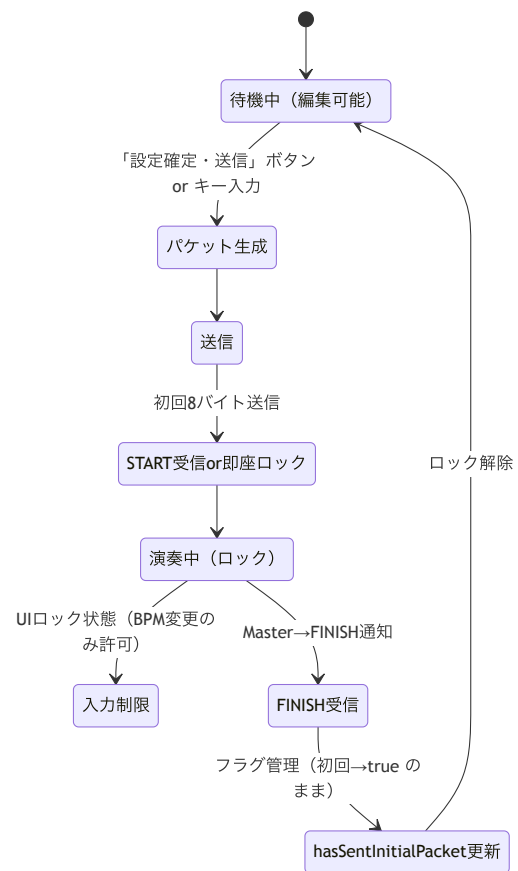


図 3 Processing から Master への送信の状態遷移図

2 担当箇所の詳細設計

2.1 ソフトウェア設計

2.1.1 全体の処理フロー

システム全体のフローチャートを図 4 に示す。本処理は、Processing の起動時に `setup()` が実行され、シリアルポートを 9600bps で初期化するところから開始する。接続に成功した場合は `isSerialConnected` を `true` に設定し、失敗した場合はシミュレーションモードとして `isSerialConnected=false` のまま UI 初期化を完了する。その後は `draw()` によるメインループに入り、UI の描画とユーザー操作の受付を繰り返す。ユーザー操作は、BPM 変更、演奏順設定、オクターブ変更、送信の 4 系統に分岐する。BPM は $\uparrow/\downarrow$ キー入力により 30~180 の範囲で更新される。演奏順は 1~3 キー入力により設定されるが、`isPlaying=true` の間に変更できず、待機中のみ `playOrder[]` へ反映される。オクターブもボタンクリックによって -1~9 の範囲で更新されるが、同様に演奏中は操作を無効化する。送信ボタンが押されると `sendParametersToMaster()` が呼び出され、Master Arduino への送信処理へ遷移する。送信処理では、`hasSentInitialPacket` の値によって初回送信と 2 回目以降の送信を切り替える。初回送信では、オクターブ 2 桁、演奏順 3 桁、BPM 3 桁を連結した初回パケットを `buildPacket()` で生成し、シリアル接続時は Master Arduino へ送信する。送信完了後は `hasSentInitialPacket` を `true` に更新し、`isPlaying=true` として UI をロックする。2 回目以降は BPM のみを 3 桁で送信し、演奏中のテンポ変更に対応する。また、Master Arduino からの状態通知は `serialEvent()` で受信する。START または PLAYING を受信した場合は `isPlaying=true` を維持してロック状態を継続し、FINISH または STOP を受信した場合は `isPlaying=false` としてロックを解除する。これにより、演奏中の設定変更を防ぎつつ、演奏終了後は再び設定入力可能な状態へ戻る。

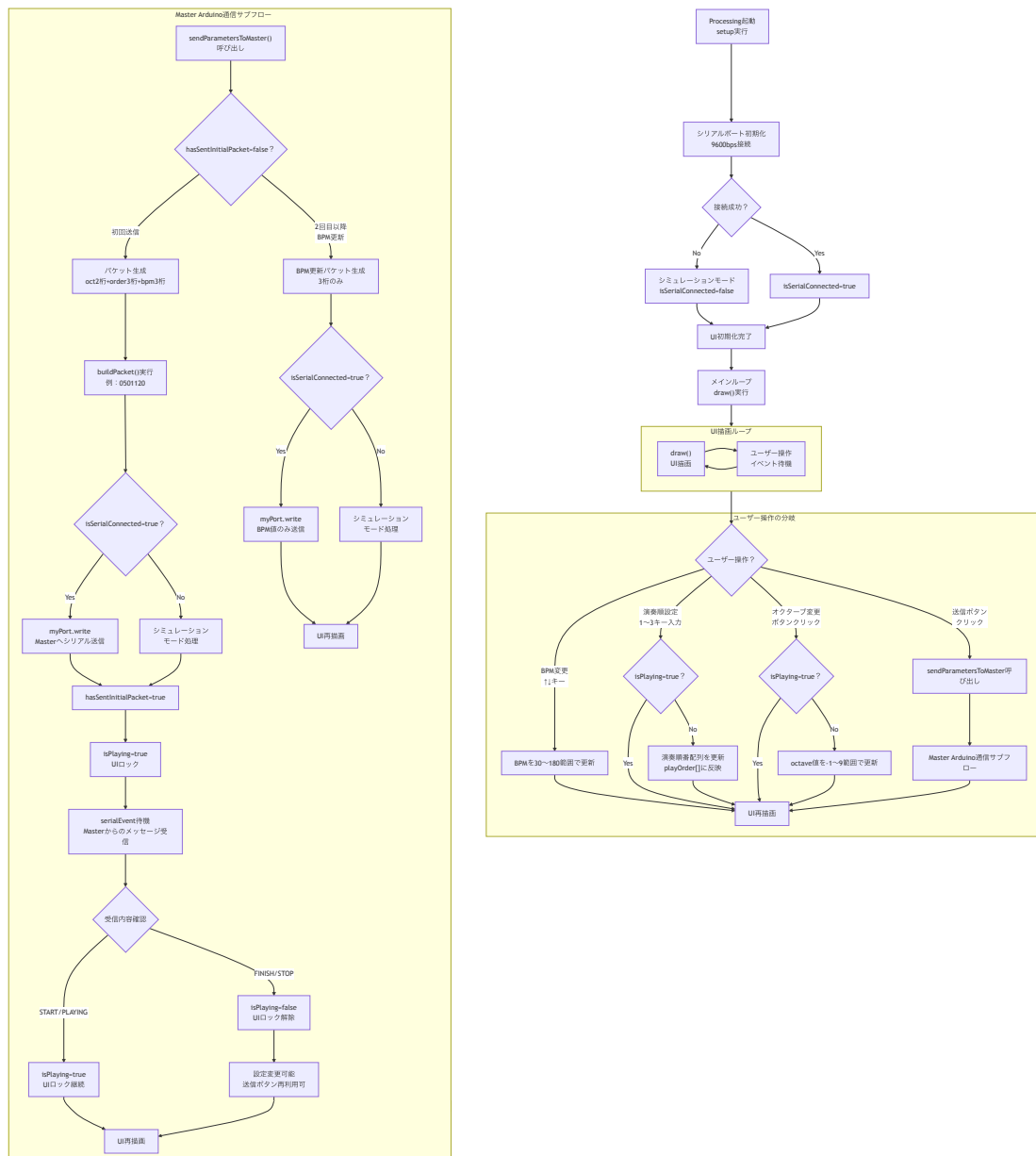


図 4 UI および Master Arduino への送信制御フローチャート